

Dưới đây là mã nguồn đã được nâng cấp, bổ sung đầy đủ **2 tính năng mới**:

1. **Vẽ Bounding Box kéo thả tự do**: Đè giữ và kéo chuột trái để tạo khung bounding box có kích thước tùy ý.
2. **Tự động lưu & xuất file classes.txt**: Tự động ghi danh sách tên nhãn (theo thứ tự class_id) vào file classes.txt tại thư mục **Output** mỗi khi thêm nhãn mới hoặc thực hiện lưu nhãn.

Mã nguồn hoàn chỉnh (PyQt5)

```
import sys
import os
import math
from PyQt5.QtCore import Qt, QRectF, QPointF
from PyQt5.QtGui import QPixmap, QPen, QColor, QFont
from PyQt5.QtWidgets import (
    QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
    QPushButton, QFileDialog, QListWidget, QInputDialog, QLabel,
    QMessageBox
)

class LabelingCanvas(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setMouseTracking(True)
        self.setFocusPolicy(Qt.StrongFocus)

        # Trạng thái dữ liệu
        self.image_path = ""
        self.pixmap = None
        self.scale = 1.0
        self.offset = QPointF(0, 0)

        # Danh sách box: dict(class_id, x_center, y_center, width,
height)
        self.bboxes = []

        # Danh sách nhãn: dict(id, name)
        self.classes = []
        self.current_class_id = 0

        # Kích thước box nhấp (mặc định cho mode gợi ý)
        self.draft_width = 100
        self.draft_height = 100

        # Trạng thái tương tác chuột
        self.mode = "suggest" # "suggest" (gợi ý/vẽ kéo thả) hoặc
"select" (chọn/sửa)
        self.alt_pressed = False
```

```

        self.ctrl_pressed = False
        self.is_drawing_drag = False # Đang trong trạng thái kéo
chuột để vẽ box tự do

        self.selected_box_idx = -1
        self.resizing_handle = None # 'tl' (top-left), 'br'
(bottom-right)
        self.drag_start_img_pos = QPointF()

    def set_image(self, image_path, boxes=None):
        self.image_path = image_path
        self.pixmap = QPixmap(image_path)
        self.bboxes = bboxes if bboxes is not None else []
        self.selected_box_idx = -1
        self.scale = 1.0
        self.offset = QPointF(0, 0)
        self.is_drawing_drag = False
        self.update()

    def img_to_canvas(self, point):
        return QPointF(point.x() * self.scale + self.offset.x(),
                        point.y() * self.scale + self.offset.y())

    def canvas_to_img(self, point):
        return QPointF((point.x() - self.offset.x()) / self.scale,
                        (point.y() - self.offset.y()) / self.scale)

    def paintEvent(self, event):
        from PyQt5.QtGui import QPainter
        painter = QPainter(self)
        painter.setRenderHint(QPainter.Antialiasing)

        # 1. Vẽ ảnh
        if self.pixmap:
            scaled_w = self.pixmap.width() * self.scale
            scaled_h = self.pixmap.height() * self.scale
            target_rect = QRectF(self.offset.x(), self.offset.y(),
scaled_w, scaled_h)
            painter.drawPixmap(target_rect, self.pixmap,
QRectF(self.pixmap.rect()))

        if not self.pixmap:
            return

        img_w, img_h = self.pixmap.width(), self.pixmap.height()

        # 2. Vẽ các Box đã định nghĩa
        for idx, b in enumerate(self.bboxes):

```

```

cx, cy = b['x_center'] * img_w, b['y_center'] * img_h
bw, bh = b['width'] * img_w, b['height'] * img_h

x1 = (cx - bw/2) * self.scale + self.offset.x()
y1 = (cy - bh/2) * self.scale + self.offset.y()
w_c = bw * self.scale
h_c = bh * self.scale

rect = QRectF(x1, y1, w_c, h_c)
is_selected = (idx == self.selected_box_idx)

pen = QPen(QColor(255, 0, 0) if is_selected else QColor(0,
255, 0), 2)
painter.setPen(pen)
painter.drawRect(rect)

# Vẽ tên Class
class_name = str(b['class_id'])
for c in self.classes:
    if c['id'] == b['class_id']:
        class_name = c['name']
        break
painter.setPen(QPen(QColor(255, 255, 255)))
painter.setFont(QFont("Arial", 10, QFont.Bold))
painter.drawText(int(x1), int(y1 - 5), f"[{b['class_id']}]
{class_name}")

# Vẽ điểm neo kéo thả ở góc (nếu chọn)
if is_selected:
    painter.setBrush(QColor(0, 0, 255))
    painter.drawRect(QRectF(x1 - 4, y1 - 4, 8, 8)) #
Top-Left (TL)
    painter.drawRect(QRectF(x1 + w_c - 4, y1 + h_c - 4, 8,
8)) # Bottom-Right (BR)

# 3. Vẽ Box nhấp gọi ý hoặc Khung xem trước khi kéo thả chuột
if self.mode == "suggest" and self.mousePressEvent():
    cursor_pos = self.mapFromGlobal(self.cursor().pos())
    painter.setPen(QPen(QColor(255, 255, 0), 1.5,
Qt.DashLine))

    if self.is_drawing_drag:
        # Đang đề giữ chuột để vẽ khung tự do
        start_c = self.img_to_canvas(self.drag_start_img_pos)
        rect_drag = QRectF(start_c, cursor_pos).normalized()
        painter.drawRect(rect_drag)
    else:
        # Đang rê chuột (hiển thị khung nhấp hình chữ nhật cố

```

định kích thước)

```
        bw_c = self.draft_width * self.scale
        bh_c = self.draft_height * self.scale
        x1 = cursor_pos.x() - bw_c / 2
        y1 = cursor_pos.y() - bh_c / 2

        painter.drawRect(QRectF(x1, y1, bw_c, bh_c))
        painter.setPen(QPen(QColor(255, 255, 0)))
        painter.drawPoint(cursor_pos)

def mousePressEvent(self, event):
    if not self.pixmap:
        return

    pos = event.pos()
    img_pos = self.canvas_to_img(pos)
    img_w, img_h = self.pixmap.width(), self.pixmap.height()

    if event.button() == Qt.LeftButton:
        if self.mode == "suggest":
            # Bắt đầu kéo thả vẽ box mới
            self.is_drawing_drag = True
            self.drag_start_img_pos = img_pos
            self.update()

        elif self.mode == "select":
            # Kiểm tra kéo thả góc box đang được chọn
            if self.selected_box_idx >= 0:
                b = self.bboxes[self.selected_box_idx]
                cx, cy = b['x_center'] * img_w, b['y_center'] *
img_h

                bw, bh = b['width'] * img_w, b['height'] * img_h
                x1, y1 = cx - bw/2, cy - bh/2
                x2, y2 = cx + bw/2, cy + bh/2

                threshold = 15 / self.scale
                dist_tl = math.hypot(img_pos.x() - x1, img_pos.y()
- y1)

                dist_br = math.hypot(img_pos.x() - x2, img_pos.y()
- y2)

                if dist_tl < threshold:
                    self.resizing_handle = 'tl'
                    return
                elif dist_br < threshold:
                    self.resizing_handle = 'br'
                    return
```

```

        # Chọn box khác
        self.selected_box_idx = -1
        for idx, b in enumerate(self.bboxes):
            cx, cy = b['x_center'] * img_w, b['y_center'] *
img_h

            bw, bh = b['width'] * img_w, b['height'] * img_h
            rect = QRectF(cx - bw/2, cy - bh/2, bw, bh)
            if rect.contains(img_pos):
                self.selected_box_idx = idx
                break
        self.update()

def mouseMoveEvent(self, event):
    if self.mode == "suggest":
        self.update()

        elif self.mode == "select" and self.resizing_handle and
self.selected_box_idx >= 0:
        img_pos = self.canvas_to_img(event.pos())
        img_w, img_h = self.pixmap.width(), self.pixmap.height()

        b = self.bboxes[self.selected_box_idx]
        cx, cy = b['x_center'] * img_w, b['y_center'] * img_h
        bw, bh = b['width'] * img_w, b['height'] * img_h
        x1, y1 = cx - bw/2, cy - bh/2
        x2, y2 = cx + bw/2, cy + bh/2

        if self.resizing_handle == 'tl':
            # Giữ nguyên góc Bottom-Right (x2, y2)
            x1_new, y1_new = img_pos.x(), img_pos.y()
            new_w = max(5, x2 - x1_new)
            new_h = max(5, y2 - y1_new)
            new_cx = x2 - new_w / 2
            new_cy = y2 - new_h / 2
        else: # 'br'
            # Giữ nguyên góc Top-Left (x1, y1)
            x2_new, y2_new = img_pos.x(), img_pos.y()
            new_w = max(5, x2_new - x1)
            new_h = max(5, y2_new - y1)
            new_cx = x1 + new_w / 2
            new_cy = y1 + new_h / 2

        b['x_center'] = new_cx / img_w
        b['y_center'] = new_cy / img_h
        b['width'] = new_w / img_w
        b['height'] = new_h / img_h
        self.update()

```

```

def mousePressEvent(self, event):
    if event.button() == Qt.LeftButton and self.mode == "suggest"
and self.is_drawing_drag:
        self.is_drawing_drag = False
        img_pos = self.canvas_to_img(event.pos())
        img_w, img_h = self.pixmap.width(), self.pixmap.height()

        x1 = min(self.drag_start_img_pos.x(), img_pos.x())
        y1 = min(self.drag_start_img_pos.y(), img_pos.y())
        x2 = max(self.drag_start_img_pos.x(), img_pos.x())
        y2 = max(self.drag_start_img_pos.y(), img_pos.y())

        w = x2 - x1
        h = y2 - y1

        # Nếu kích thước kéo thả rất nhỏ -> được tính là 1 click
ĐẶT TÂM BOX NHẤP
        if w < 5 or h < 5:
            w = self.draft_width
            h = self.draft_height
            cx = img_pos.x()
            cy = img_pos.y()
        else:
            cx = x1 + w / 2
            cy = y1 + h / 2

        # Tạo box mới
        new_box = {
            'class_id': self.current_class_id,
            'x_center': cx / img_w,
            'y_center': cy / img_h,
            'width': w / img_w,
            'height': h / img_h
        }
        self.bboxes.append(new_box)
        self.selected_box_idx = len(self.bboxes) - 1
        self.mode = "select" # Tự động chuyển qua chế độ chọn/sửa
sau khi tạo xong
        self.update()

        self.resizing_handle = None

def wheelEvent(self, event):
    delta = event.angleDelta().y()

    # 1. Thu phóng kích thước nhấn nhấp: Alt + Lăn chuột
    if self.alt_pressed:
        factor = 1.1 if delta > 0 else 0.9

```

```

        self.draft_width = max(10, self.draft_width * factor)
        self.draft_height = max(10, self.draft_height * factor)
        self.update()

# 2. Thu phóng ảnh (Zoom): Ctrl + Lăn chuột
elif self.ctrl_pressed:
    cursor_pos = event.pos()
    old_img_pos = self.canvas_to_img(cursor_pos)

    factor = 1.15 if delta > 0 else 0.85
    self.scale *= factor

    # Căn chỉnh lại offset để zoom theo vị trí con trỏ chuột
    new_x = cursor_pos.x() - old_img_pos.x() * self.scale
    new_y = cursor_pos.y() - old_img_pos.y() * self.scale
    self.offset = QPointF(new_x, new_y)
    self.update()

def keyPressEvent(self, event):
    if event.key() == Qt.Key_Alt:
        self.alt_pressed = True
        self.mode = "select" if self.mode == "suggest" else
"suggest"
        self.update()
    elif event.key() == Qt.Key_Control:
        self.ctrl_pressed = True

def keyReleaseEvent(self, event):
    if event.key() == Qt.Key_Alt:
        self.alt_pressed = False
    elif event.key() == Qt.Key_Control:
        self.ctrl_pressed = False

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("YOLO Object Detection Labeling Tool")
        self.resize(1200, 800)

        self.input_dir = ""
        self.output_dir = ""
        self.image_files = []
        self.current_img_idx = -1

        self.init_ui()

    def init_ui(self):

```

```

main_widget = QWidget()
self.setCentralWidget(main_widget)
layout = QHBoxLayout(main_widget)

# Panel bên trái: Canvas hiển thị & đánh nhãn
self.canvas = LabelingCanvas(self)
layout.addWidget(self.canvas, stretch=4)

# Panel bên phải: Điều khiển & Quản lý nhãn
right_panel = QVBoxLayout()

# Nút chọn thư mục
self.btn_input = QPushButton("Thư mục ảnh (Input)")
self.btn_input.clicked.connect(self.select_input_dir)
right_panel.addWidget(self.btn_input)

self.btn_output = QPushButton("Thư mục nhãn (Output)")
self.btn_output.clicked.connect(self.select_output_dir)
right_panel.addWidget(self.btn_output)

# Thông tin ảnh
self.lbl_info = QLabel("Chưa chọn ảnh")
right_panel.addWidget(self.lbl_info)

# Thanh quản lý ID và Tên nhãn (Classes)
right_panel.addWidget(QLabel("<b>Danh sách Nhãn  
(Classes):</b>"))
self.class_list_widget = QListWidget()

self.class_list_widget.currentRowChanged.connect(self.change_selected_
class)
right_panel.addWidget(self.class_list_widget)

btn_add_class = QPushButton("Thêm Nhãn mới")
btn_add_class.clicked.connect(self.add_class)
right_panel.addWidget(btn_add_class)

# Chọn class cho Box đang chọn
right_panel.addWidget(QLabel("<b>Gán Nhãn cho Box chọn:</b>"))
self.btn_apply_class = QPushButton("Đổi ID Class cho Box  
chọn")

self.btn_apply_class.clicked.connect(self.apply_class_to_selected_box)
right_panel.addWidget(self.btn_apply_class)

# Nút lưu & chuyển ảnh
btn_save = QPushButton("Lưu nhãn (S)")
btn_save.clicked.connect(self.save_labels)

```



```

right_panel.addWidget(btn_save)

btn_next = QPushButton("Ảnh tiếp (D / ->)")
btn_next.clicked.connect(self.next_image)
right_panel.addWidget(btn_next)

btn_prev = QPushButton("Ảnh trước (A / <-)")
btn_prev.clicked.connect(self.prev_image)
right_panel.addWidget(btn_prev)

right_panel.addStretch()
layout.addLayout(right_panel, stretch=1)

def select_input_dir(self):
    dir_path = QFileDialog.getExistingDirectory(self, "Chọn thư
mục Input chứa ảnh")
    if dir_path:
        self.input_dir = dir_path
        valid_exts = ('.jpg', '.jpeg', '.png', '.bmp')
        self.image_files = [f for f in os.listdir(dir_path) if
f.lower().endswith(valid_exts)]
        self.image_files.sort()
        if self.image_files:
            self.current_img_idx = 0
            self.load_image_data()

def select_output_dir(self):
    dir_path = QFileDialog.getExistingDirectory(self, "Chọn thư
mục Output lưu ảnh")
    if dir_path:
        self.output_dir = dir_path
        self.load_classes_txt() # Đọc file classes.txt nếu đã tồn
tại sẵn trong thư mục Output

def load_image_data(self):
    if not (0 <= self.current_img_idx < len(self.image_files)):
        return

    filename = self.image_files[self.current_img_idx]
    img_path = os.path.join(self.input_dir, filename)

    boxes = []
    # Tự động đọc nhãn .txt tương ứng nếu đã tồn tại trong Output
    if self.output_dir:
        txt_filename = os.path.splitext(filename)[0] + ".txt"
        txt_path = os.path.join(self.output_dir, txt_filename)
        if os.path.exists(txt_path):
            with open(txt_path, 'r', encoding='utf-8') as f:

```

```

        for line in f.readlines():
            parts = line.strip().split()
            if len(parts) >= 5:
                boxes.append({
                    'class_id': int(parts[0]),
                    'x_center': float(parts[1]),
                    'y_center': float(parts[2]),
                    'width': float(parts[3]),
                    'height': float(parts[4])
                })

        self.canvas.set_image(img_path, boxes)
        self.lbl_info.setText(f"Ảnh [{self.current_img_idx +
1}/{len(self.image_files)}]: {filename}")

    def save_labels(self):
        if not self.output_dir:
            QMessageBox.warning(self, "Cảnh báo", "Vui lòng chọn thư
mục Output trước khi lưu!")
            return
        if self.current_img_idx < 0:
            return

        # 1. Lưu file nhãn .txt cho ảnh hiện tại
        filename = self.image_files[self.current_img_idx]
        txt_filename = os.path.splitext(filename)[0] + ".txt"
        txt_path = os.path.join(self.output_dir, txt_filename)

        lines = []
        for b in self.canvas.boxes:
            line = f"{b['class_id']} {b['x_center']:.6f}
{b['y_center']:.6f} {b['width']:.6f} {b['height']:.6f}"
            lines.append(line)

        with open(txt_path, 'w', encoding='utf-8') as f:
            f.write("\n".join(lines))

        # 2. Xuất/lưu file classes.txt vào thư mục Output
        self.export_classes_txt()

        QMessageBox.information(self, "Thông báo", f"Đã lưu nhãn và
file classes.txt thành công!")

    def export_classes_txt(self):
        """Tự động xuất danh sách các nhãn hiện có vào file
classes.txt"""
        if not self.output_dir:
            return

```

```

        classes_path = os.path.join(self.output_dir, "classes.txt")
        sorted_classes = sorted(self.canvas.classes, key=lambda x:
x['id'])
        class_names = [c['name'] for c in sorted_classes]

        with open(classes_path, 'w', encoding='utf-8') as f:
            f.write("\n".join(class_names))

    def load_classes_txt(self):
        """Tự động đọc file classes.txt nếu đã có sẵn trong thư mục
Output"""
        if not self.output_dir:
            return
        classes_path = os.path.join(self.output_dir, "classes.txt")
        if os.path.exists(classes_path):
            self.canvas.classes.clear()
            self.class_list_widget.clear()
            with open(classes_path, 'r', encoding='utf-8') as f:
                lines = [line.strip() for line in f.readlines() if
line.strip()]
                for idx, name in enumerate(lines):
                    new_class = {'id': idx, 'name': name}
                    self.canvas.classes.append(new_class)
                    self.class_list_widget.addItem(f"[{idx}] {name}")
            if self.canvas.classes:
                self.class_list_widget.setCurrentRow(0)

    def add_class(self):
        name, ok = QDialog.getText(self, "Thêm Class", "Nhập tên
nhãn:")
        if ok and name:
            class_id = len(self.canvas.classes)
            new_class = {'id': class_id, 'name': name}
            self.canvas.classes.append(new_class)
            self.class_list_widget.addItem(f"[{class_id}] {name}")
            self.class_list_widget.setCurrentRow(class_id)
            # Tự động xuất/cập nhật classes.txt khi thêm nhãn mới
            self.export_classes_txt()

    def change_selected_class(self, row):
        if row >= 0 and row < len(self.canvas.classes):
            self.canvas.current_class_id =
self.canvas.classes[row]['id']

    def apply_class_to_selected_box(self):
        if self.canvas.selected_box_idx >= 0 and
self.class_list_widget.currentRow() >= 0:
            box = self.canvas.bboxes[self.canvas.selected_box_idx]

```

```

        box['class_id'] = self.canvas.current_class_id
        self.canvas.update()

    def next_image(self):
        if self.current_img_idx < len(self.image_files) - 1:
            if self.output_dir:
                self.save_labels()
            self.current_img_idx += 1
            self.load_image_data()

    def prev_image(self):
        if self.current_img_idx > 0:
            if self.output_dir:
                self.save_labels()
            self.current_img_idx -= 1
            self.load_image_data()

    def keyPressEvent(self, event):
        if event.key() in (Qt.Key_D, Qt.Key_Right):
            self.next_image()
        elif event.key() in (Qt.Key_A, Qt.Key_Left):
            self.prev_image()
        elif event.key() == Qt.Key_S:
            self.save_labels()
        elif event.key() == Qt.Key_Delete and
self.canvas.selected_box_idx >= 0:
            del self.canvas.bboxes[self.canvas.selected_box_idx]
            self.canvas.selected_box_idx = -1
            self.canvas.update()
        else:
            super().keyPressEvent(event)

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec_())

```

Tóm tắt cải tiến chi tiết

1. Vẽ Bounding Box Kéo Thả Tự Do (Drag & Drop):

- Đè giữ chuột trái và kéo rê chuột để chủ động chọn vùng và độ lớn khung nhãn.
- Nếu chỉ nhấp chuột đơn lẻ (không kéo rê), công cụ vẫn giữ tính năng thông minh là **đặt tâm theo khung gọi ý nhấp**.

2. Quản lý & Tự Động Xuất File classes.txt:

- Ngay khi bạn nhấn "**Thêm Nhãn mới**" hoặc nhấn nút "**Lưu nhãn (S)**", phần mềm tự động tạo/cập nhật file classes.txt tại thư mục Output.

- Đảm bảo cấu trúc chuẩn YOLO: Mỗi dòng ứng với tên nhãn theo đúng thứ tự Class ID (0, 1, 2,...).
- Khi chọn thư mục **Output** đã có sẵn file classes.txt, ứng dụng sẽ tự động tải danh sách nhãn vào giao diện.